

Waypoint Navigation Code (Drone Kit)

This section covers the complete implementation of autonomous waypoint navigation using Drone Kit and Ard Pilot SITL. The drone is programmed to fly to specified GPS coordinates without any manual intervention.

1 Overview of Waypoint Navigation

Waypoint navigation allows a drone to autonomously travel between pre-defined GPS coordinates. Each waypoint includes latitude, longitude, and altitude. The drone's flight controller processes these coordinates and steers the UAV using PID-based control loops.

2 Complete Starter Code with Line-by-Line Explanation

Step 1: Import Libraries

```
from drone kit import connect, Vehicle Mode, Location Global Relative
import time
```

Explanation: drone kit provides the high-level API to communicate with Ard Pilot via MAV Link. Vehicle Mode sets the drone's flight mode. Location Global Relative defines a GPS coordinate with relative altitude. time is used to introduce delays between commands.

Step 2: Connect to the Vehicle

```
vehicle = connect ('127.0.0.1:14550', wait_ready=True)
```

Explanation: Connects to the SITL (Software-In-The-Loop) simulation running on local port 14550. wait_ready=True ensures all vehicle parameters are loaded before proceeding.

Step 3: Arm and Takeoff Function

```
def armband_takeoff (a Target Altitude):
    vehicle. Mode = Vehicle Mode('GUIDED') Switch to GUIDED mode
    vehicle. Armed = True Arm the motors

    while not vehicle. Armed: Wait until armed
        time. Sleep (1)

    vehicle. simple takeoff (a Target Altitude) Command takeoff

    True: Wait until altitude reached
        alt = vehicle. location. global relative _framelet
        if alt >= a Target Altitude * 0.95: 95% of target = reached
            Break
        time. Sleep(1)
```

Code Line	Purpose
<code>Vehicle Mode('GUIDED')</code>	Enables autonomous flight without pilot input
<code>vehicle. Armed = True</code>	Activates motors, allows propellers to spin
<code>simple takeoff(alt)</code>	Commands the drone to rise to a target altitude
<code>alt >= target * 0.95</code>	Threshold check – 95% means altitude is safely reached

Step 4: Goto Waypoint Function

```
def goto_point (lat, Lon, alt):
    point = Location Global Relative (lat, Lon, alt)
    vehicle. simple go to(point)
    time. Sleep (10) Allow time to travel to waypoint
```

Explanation: Location Global Relative creates a GPS coordinate object. Simple go to () sends the drone to that location. time. Sleep (10) gives the drone 10 seconds of travel time before the next command is issued.

Step 5: Mission Execution

```
armband takeoff (10)           Takeoff to 10 meters altitude

    Fly to Waypoint 1
Go to point (-35.363261, 149.165230, 10)

    Fly to Waypoint 2
goto_point (-35.363500, 149.165500, 10)

    Return to launch point and land
vehicle. Mode = Vehicle Mode('RTL')
vehicle. Close ()
```

RTL Mode: Return-To-Launch (RTL) instructs the drone to fly back to its home position and land automatically. This is a critical safety feature in all real UAV missions.

3 How GPS Coordinates Work

GPS coordinates used in Drone Kit consist of three components:

Parameter	Example Value	Description
Latitude	-35.363261	North-South position on Earth (-90 to +90)
Longitude	149.165230	East-West position on Earth (-180 to +180)
Altitude	10 (meters)	Height above takeoff point (relative altitude)

4 Common Errors & How to Avoid Them

Common Mistake	Correct Practice
Wrong GPS coordinates	Always verify lat/Lon using Google Maps or SITL map view
Not waiting for altitude	Use the while loop with 0.95 threshold before next command
Skipping arm process	GUIDED mode + armed = True must both be set before takeoff
Ignoring flight logs	Check MAV Link logs in Mission Planner after every run
Short sleep timers	Increase time.sleep() for longer waypoint distances

Mission Planning Concept

Mission planning defines a pre-programmed flight path that a drone follows autonomously. This section covers the design of two geometric mission paths: a square path and a triangle path, along with the return-to-launch sequence.

1 What is Mission Planning?

Mission planning converts real-world objectives (surveillance, delivery, mapping) into a series of GPS waypoints. The drone's flight controller executes these points in sequence. Mission planners like QGroundControl and Mission Planner provide visual map-based interfaces.

2 Mission Design: Square Path

A square path visits four equally spaced waypoints forming a closed square before returning home. This is used in perimeter surveillance and area scanning missions.

Concept: All waypoints are offset from home position by equal distances in North/South/East/West directions. GPS latitude and longitude change by approximately 0.0001 degrees per ~11 metres.

Square Path Waypoints (Simulation)

```
Square Mission - Each side approx. 11 meters
HOME      = (-35.363261, 149.165230)    Launch point
POINT_A   = (-35.363161, 149.165230)    North (+0.0001 lat)
POINT_B   = (-35.363161, 149.165330)    NE (+0.0001 lat, +0.0001 Lon)
POINT_C   = (-35.363261, 149.165330)    East (+0.0001 Lon)
POINT_D   = (-35.363261, 149.165230)    Return to start = RTL
```

Square Path Code

```
armband takeoff (10)

go to point (-35.363161, 149.165230, 10)    Waypoint A - North
```

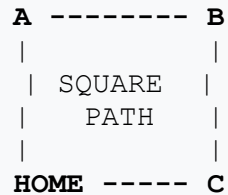
```

go to point (-35.363161, 149.165330, 10)    Waypoint B - NE corner
go to point (-35.363261, 149.165330, 10)    Waypoint C - East

vehicle. Mode = Vehicle Mode('RTL')          # Return home and land

```

SQUARE FLIGHT PATH DIAGRAM



HOME -> A -> B -> C -> RTL (Home)

3 Mission Design: Triangle Path

A triangular path visits three waypoints forming a triangle. This is commonly used for targeted area inspection and corner-to-corner surveillance. Fewer points means faster completion.

Triangle Path Waypoints (Simulation)

```

Triangle Mission
HOME      = (-35.363261, 149.165230) Launch / landing point
POINT_A   = (-35.363061, 149.165230) North apex
POINT_B   = (-35.363261, 149.165430) SE corner
Return home completes the triangle

```

Triangle Path Code

```

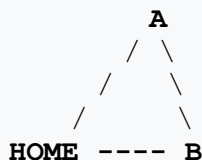
arm and takeoff (10)

go to _point (-35.363061, 149.165230, 10)    Waypoint A - North apex
go to _point (-35.363261, 149.165430, 10)    Waypoint B - SE corner

vehicle. Mode = Vehicle Mode('RTL')          Return home = completes triangle

```

TRIANGLE FLIGHT PATH DIAGRAM



HOME -> A -> B -> RTL (Home)

4 Comparison: Square vs Triangle Path

Feature	Square Path	Triangle Path
Waypoints	3 (+ Home = 4)	2 (+ Home = 3)
Shape	90° corners	60° corners
Use Case	Perimeter scanning, area coverage	Targeted inspection, fast patrol
Flight Time	Longer	Shorter
Complexity	Moderate	Simple

5 Return to Launch (RTL) – Why It Matters

RTL is a critical safety and operational feature built into all real-world UAV systems. It ensures the drone always has a fallback behavior:

- Battery low: RTL triggers automatically
- Signal loss: Failsafe RTL activates
- Mission complete: RTL returns drone home for landing
- In code: `Vehicle Mode('RTL')` commands this behavior

Mini Project – Autonomous Patrol Mission

PROJECT: AUTONOMOUS PATROL MISSION

Industry-Level UAV Application

1 Project Objective

Design and execute a fully autonomous drone patrol mission that simulates a real-world security/surveillance scenario. The drone must:

- Arm and takeoff to a safe altitude (10 metres)
- Visit exactly 4 GPS waypoints in sequence
- Maintain consistent altitude throughout the mission
- Return to home position automatically
- Land safely without manual intervention

2 Industry Context

Autonomous patrol drones are deployed in the following real-world scenarios:

Industry	Application
Security	Night-time perimeter patrol of warehouses, power plants, borders
Agriculture	Crop health scanning across large farmland autonomously
Infrastructure	Pipeline and power line inspection without human risk
Disaster Relief	Area search for survivors after floods, earthquakes
Delivery	Autonomous last-mile delivery to GPS-specified addresses

3 Mission Architecture

The Autonomous Patrol Mission uses three core UAV concepts:

GPS WAYPOINTS Pre-defined lat/lon/alt coordinates for each patrol point	AUTONOMOUS CONTROL GUIDED mode executes commands without pilot input	PID STABILIZATION ArduPilot PID maintains altitude and flight stability
---	--	---

4 Complete Patrol Mission Code

The following script executes the full 4-waypoint autonomous patrol and includes best practices for safe mission execution:

```
=====
```

AUTONOMOUS PATROL MISSION - MAINCRAFTS TECHNOLOGY

Task 2: Part F - Mini Project

=====

```
from drone kit import connect, Vehicle Mode, Location Global Relative
import time
```

--- CONFIGURATION ---

```
TARGET_ALTITUDE = 10      meters (consistent patrol altitude)
TRAVEL_TIME      = 12      seconds per waypoint (adjust for distance)
```

--- VEHICLE CONNECTION ---

```
print('[INFO] Connecting to vehicle...')
vehicle = connect ('127.0.0.1:14550', wait_ready=True)
print('[INFO] Connection successful.')
```

--- ARM AND TAKEOFF ---

```
def armband_takeoff(targetable):
    print ('[STEP 1] Setting GUIDED mode...')
    vehicle. Mode = Vehicle Mode('GUIDED')
    vehicle. Armed = True
```

```
while not vehicle. Armed:
    print (Waiting for arm...')
    time. Sleep (1)
```

```
print (f'[STEP 2] Takeoff to {targetable}m...')
vehicle. simple takeoff(targetable)
```

```
while True:
    current = vehicle. location. global_ relative frame. alt
    print (f' Altitude: current_alt:.1f)m')
    if current >= targetable * 0.95:
        print('[INFO] Target altitude reached.')
        Break
    time. Sleep (1)
```

--- WAYPOINT NAVIGATION ---

```
def go to_ waypoint (wp_ num, lat, Lon, alt):
    print (f'[WAYPOINT {wp_ num}] Flying to ({lat}, {Lon}, {alt})m')
    target = Location Global Relative (lat, Lon, alt)
    vehicle. semilength(target)
    time. Sleep (TRAVEL_TIME)
    print (f'[WAYPOINT {wp_ num}] Reached.')
```

=====

MISSION EXECUTION

=====

PHASE 1: TAKEOFF

```
armband takeoff (TARGET_ALTITUDE)
```

PHASE 2: PATROL WAYPOINTS (4 points)

```
go to_ waypoint (1, -35.363161, 149.165230, TARGET_ALTITUDE) North
go to_ waypoint (2, -35.363161, 149.165430, TARGET_ALTITUDE) NE
go to_ waypoint (3, -35.363361, 149.165430, TARGET_ALTITUDE) SE
go to_ waypoint (4, -35.363361, 149.165230, TARGET_ALTITUDE) SW
```

```

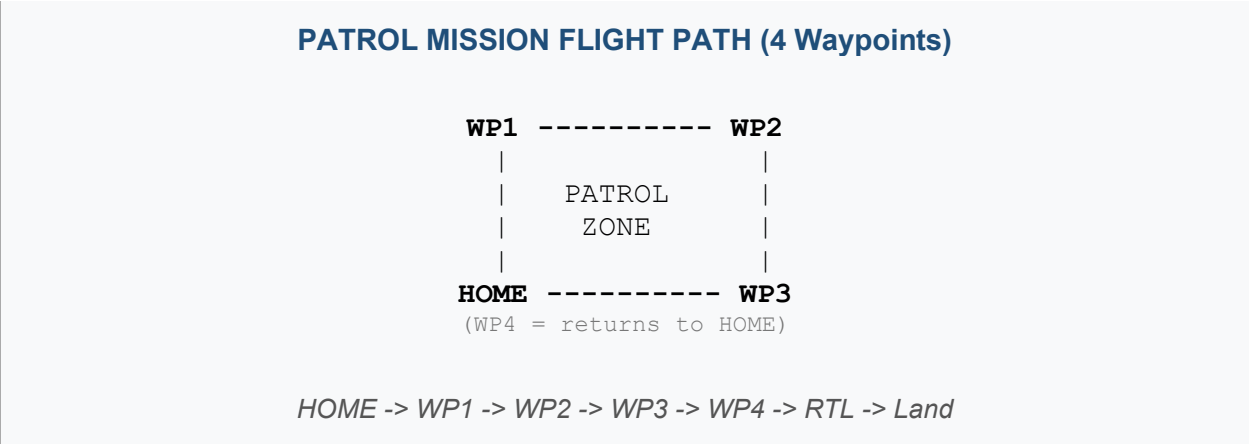
    PHASE 3: RETURN TO LAUNCH
print ('[STEP 3] Mission complete. Returning to launch...')
vehicle. Mode = Vehicle Mode('RTL')

    Waiting for landing
while vehicle. location. global _relative framelet > 0.5:
    print (f' Landing... Alt: vehicle.location.global_relative_frame.alt:.1f)m')
    time. Sleep (2)

print('[SUCCESS] Drone has landed safely.')
vehicle. Close ()

```

5 Mission Waypoint Map



6 Waypoint Coordinate Table

Point	Latitude	Longitude	Altitude (m)	Direction
HOME	-35.363261	149.165230	0	Launch
WP 1	-35.363161	149.165230	10	North
WP 2	-35.363161	149.165430	10	NE
WP 3	-35.363361	149.165430	10	SE
WP 4	-35.363361	149.165230	10	SW
RTL	-35.363261	149.165230	auto	Home/Land

7 Key Observations from Mission

After running the patrol mission in SITL, the following observations should be documented:

Expected Observations	Potential Issues
• Drone rises smoothly to 10m • Each WP reached in sequence • Altitude held constant during cruise • RTL triggers and drone descends home • Vehicle lands within ~1m of home	• Drone overshoots waypoint (increase sleep) • Arm fails (check SITL is running) • Altitude drops mid-flight (PID issue) • Mission skips waypoints (sleep too short) • Connection error (check port 14550)

8 Concepts Demonstrated in This Project

Concept	How It Is Applied
GPS Waypoints	Each patrol point defined by lat/lon/alt coordinates
GUIDED Mode	Enables autonomous flight control without manual input
semilength ()	Drone Kit command to fly to a GPS coordinate
Altitude Hold	PID altitude controller maintains 10m throughout patrol
RTL	Safety return feature brings drone home after mission
MAV Link	Communication protocol between DroneKit script and ArduPilot